1. “Summarize this agreement”

Suppose the user uploads a **rental agreement PDF**.

The document may contain 10–20 pages with information such as:

- Property address
- Property area
- Monthly rent
- Security deposit
- Lease period
- Notice period
- Maintenance responsibilities
- Tenant responsibilities
- Owner responsibilities
- Penalties
- Renewal conditions

Instead of asking the user to read the entire document, your RAG system retrieves the important sections and asks the LLM to produce a simple summary.

Example:

User uploads:

`rental_agreement.pdf`

User asks:

Summarize this agreement.

Your system retrieves relevant chunks and generates:

Agreement Summary

- Property: 2BHK apartment
- Lease period: 11 months
- Monthly rent: ₹25,000
- Security deposit: ₹1,00,000

- Notice period: 2 months
- Maintenance: Tenant is responsible for monthly maintenance charges.
- Repairs: Major structural repairs are the owner's responsibility.
- Renewal: Agreement can be renewed with mutual consent.

So the important point is:

PDF → Extract text → Split into chunks → Retrieve relevant information → LLM → Summary

2. “Compare Property A and Property B”

This is a more advanced feature of your RAG project.

Instead of uploading only one document, the user uploads **two property documents**.

For example:

Property A

- Area: 1,200 sq.ft
- Rent: ₹25,000
- Deposit: ₹1,00,000
- Maintenance: ₹3,000
- Lease: 11 months

Property B

- Area: 1,500 sq.ft
- Rent: ₹30,000
- Deposit: ₹1,50,000
- Maintenance: ₹2,000
- Lease: 12 months

The user asks:

Compare Property A and Property B.

Your RAG system retrieves information from **both documents** and gives a structured comparison.

Feature	Property A	Property B
Area	1,200 sq.ft	1,500 sq.ft
Rent	₹25,000	₹30,000
Deposit	₹1,00,000	₹1,50,000
Maintenance	₹3,000	₹2,000
Lease	11 months	12 months

Then the LLM can explain:

Property B provides a larger area and a longer lease period, but it has a higher monthly rent and security deposit. Property A is more affordable overall.

How your RAG handles this

You can store documents with metadata:

```
Property_A
    ├── property_details.pdf
    └── rental_agreement.pdf

Property_B
    ├── property_details.pdf
    └── rental_agreement.pdf
```

When the user asks for comparison:

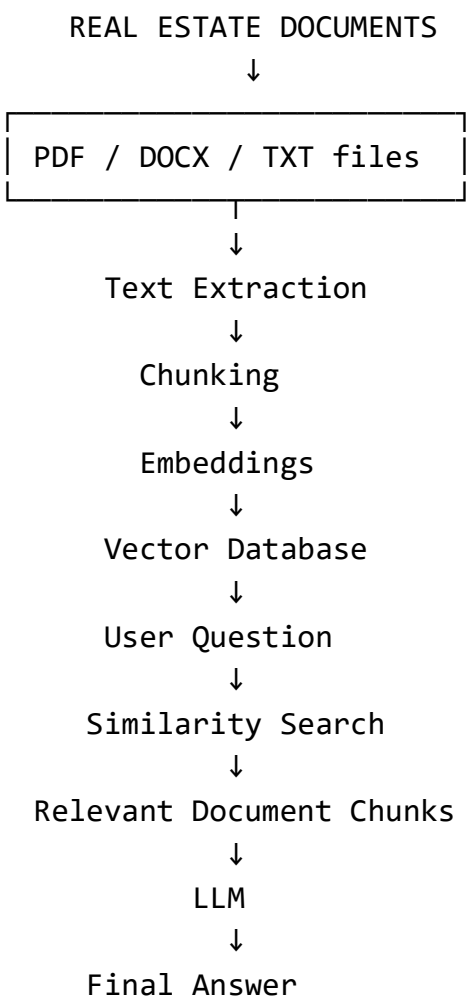
```
Question
   ↓
Retrieve Property A information
   +
Retrieve Property B information
   ↓
Send both contexts to LLM
   ↓
Generate comparison
```

This is more impressive than a normal chatbot because the system is actually **reasoning over information from multiple documents**.

3. “This gives you a strong document-based RAG project”

This means your project isn't just a simple chatbot.

You are demonstrating an actual **RAG pipeline**:



For your project, you can have **three major capabilities**:

1. Document Q&A

"What is the security deposit?"

→ ₹1,00,000

2. Document Summarization

"Summarize this agreement."

→ Gives a short summary of the important terms.

3. Multi-document Comparison

"Compare Property A and Property B."

→ Retrieves information from both documents and creates a comparison.

Why this is strong for your resume

You can say:

Real Estate Document Intelligence – RAG System

Developed a document-based RAG application that extracts information from real-estate documents, performs semantic retrieval, summarizes rental agreements, and compares multiple property documents using embeddings, vector search, and an LLM.

The **best implementation order for you as a beginner** is:

Step 1: Upload PDF

→ **Step 2:** Extract text

→ **Step 3:** Split text into chunks

→ **Step 4:** Create embeddings

→ **Step 5:** Store embeddings in a vector database

→ **Step 6:** Ask questions about one document

→ **Step 7:** Add “Summarize Agreement”

→ **Step 8:** Add Property A vs Property B comparison

→ **Step 9:** Build the frontend.

That way, you first understand **basic RAG**, then add the two advanced features: **summarization + multi-document comparison**.